

# 회귀

## ✓01 회귀 소개

### 🔧 회귀 개념

- 회귀 분석은 데이터 값이 평균과 같은 일정한 값으로 돌아가려는 경향을 이용한 통계학 기법
- 통계학 용어로는 회귀는 여러 개의 독립변수와 한 개의 종속변수 간의 상관관계를 모델링하는 기법
- 식:

$$Y = W1 \cdot X1 + W2 \cdot X2 + \dots + Wn \cdot Xn$$

Y = 종속변수

X1, X2, X3, ..., Xn = 독립변수

W1, W2, W3, ..., Wn = 독립변수의 값에 영향을 미치는 회귀 계수(Regression coefficients)

### 🔧 머신러닝 관점:

- 독립변수 = 피쳐, 종속변수 = 결정 값
- 머신러닝 회귀 예측의 핵심 : 주어진 피쳐와 결정 값 데이터 기반에서 학습을 통해 최적의 회귀 계수를 찾아내는 것

### 🔧 회귀 유형

| 독립변수 개수     | 회귀 개수의 결합   |
|-------------|-------------|
| 1개: 단일 회귀   | 선형: 선형 회귀   |
| 여러 개: 다중 회귀 | 비선형: 비선형 회귀 |

### 🔧 지도학습의 주요 두 가지 유형

- 분류 : 예측값이 카테고리나 같은 이산형 클래스 값
- 회귀 : 연속형 숫자 값

### 🔧 선형 회귀

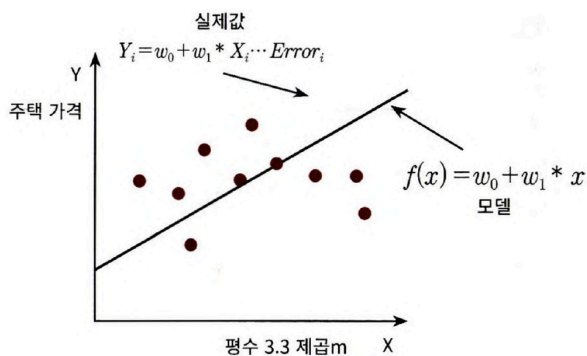
- 가장 많이 사용되는 회귀
- 실제 값과 예측값의 차이를 최소화하는 직선형 회귀선을 최적화하는 방식
- 규제(Regularization) 방법에 따라 나눌 수 있는 유형들 (규제: 과적합 문제 해결 위해 회귀 계수에 패널티 값을 적용하는 것)
  - 일반 선형 회귀 : 예측값과 실제 값의 RSS를 최소화할 수 있도록 회귀 계수를 최적화, 규제 적용 안함
  - 릿지(Ridge): 선형 회귀에 L2 규제를 추가한 모델 (L2 규제: 큰 회귀 계수 값의 예측 영향도를 감소시키기 위해 회귀 계수값을 더 적게 만드는 규제)

- 라쏘(Lasso): 선형 회귀에 L1 규제를 적용한 방식 (L1 규제: 예측 영향력이 작은 피처의 회귀 계수를 0으로 만들어 예측시 피처가 선택되지 않게 함)
- 엘라스틱넷(ElasticNet): L2, L1 규제를 함께 결합한 모델
- 로지스틱 회귀(Logistic Regression): 회귀라는 이름은 붙어 있지만, 사실 분류에 사용되는 선형 모델

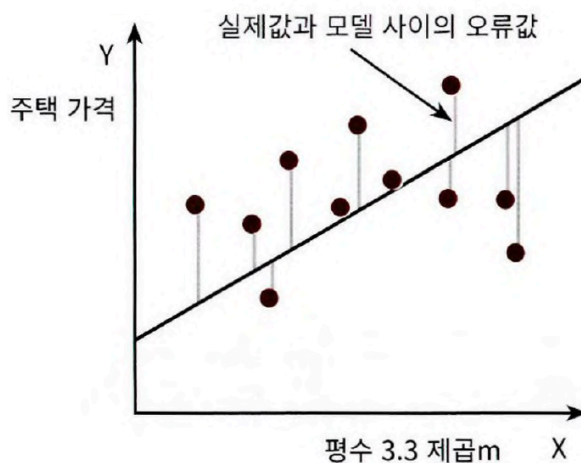
## ✓ 02 단순 선형 회귀를 통한 회귀 이해

### 🔧 단순 선형 회귀

- 독립변수도 하나, 종속변수도 하나인 선형 회귀



- 회귀 모델 함수 :  $y = w_0 + w_1 * X$ 
  - 회귀 계수 :  $w_0, w_1$  ( $w_0$  : 절편 (intercept),  $w_1$  : 기울기 (slope))
- 실제 값 = 함수 값 (-) or (+) 오류값



- 최적의 회귀 모델을 만든다는 것은 전체 데이터의 오류 값 합이 최소가 되는 모델을 만드는 것 or 오류 값 합이 최소가 될 수 있는 최적의 회귀 계수를 찾는 것
- 오류합을 구하는 방식

- 오류값은 + 나 - 가 될 수 있기 때문에 보통 절대값을 취해서 더하거나(Mean Absolute Error), 오류 값의 제곱을 구해서 더하는 방식(RSS)을 취한다.

### RSS(Residual Sum of Square)

- RSS Formula:

$$RSS(w_0, w_1) = \frac{1}{N} \sum_{i=1}^N (y_i - (w_0 + w_1 * x_i))^2$$

(i는 1부터 학습 데이터의 총 건수 N까지)

- $y_i$  : 실제값
- $w_0 + w_1 * x_i$  : 예측값
- N : 데이터 건수
- 모든 데이터의 실제값과 예측값 차이를 찾아서 데이터 건수로 나누고 평균 RSS 구하기
- RSS를 최소로 하는 회귀 계수( $w_0, w_1$ )을 학습을 통해서 찾는 게 회귀의 핵심
- RSS : 비용(Cost)함수 or 손실 함수(loss function)
- 머신러닝 회귀 알고리즘은 데이터를 계속 학습하면서 비용 함수가 반환하는 값(오류 값)을 지속해서 감소 시키고 최종적으로 더 이상 감소하지 않는 최소의 오류 값을 구하는 것

## 03 비용 최소화하기 - 경사 하강법(Gradient Descent) 소개

### 비용 최소화를 위한 경사 하강법의 역할

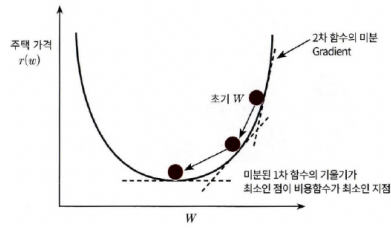
- 고차원 방정식으로는 w 파라미터가 많을 때 비용 함수 최소화가 어려움.
- 경사 하강법은 점진적으로 w를 업데이트하여 비용 함수(RSS)를 최소화하는 직관적 방법.
- 머신러닝의 학습 개념을 가능하게 한 핵심 기법.

### 경사 하강법의 원리

- 반복적으로 w를 보정하며 오류를 점진적으로 줄여 최적의 w를 찾음.
- 예: 깜깜한 밤 산 정상에서 낮은 방향으로 계속 내려가듯 오류 감소 방향으로 이동.
- 오류 값이 더 이상 줄어들지 않으면 최적 w로 결정.

### 경사 하강법과 미분

- 가속도-속도 관계처럼, 비용 함수가 포물선 형태이면, 미분 값(기울기)이 0인 지점이 최소.
- 미분을 적용하여 w를 계속 업데이트, 기울기가 0에 가까워질 때 멈춤.



## 🔧 경사 하강법 수식

- 편미분 사용:

$$\frac{\partial R(w)}{\partial w_1} = \frac{2}{N} \sum_{i=1}^N -x_i * (y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N x_i * (\text{실제값}_i - \text{예측값}_i)$$

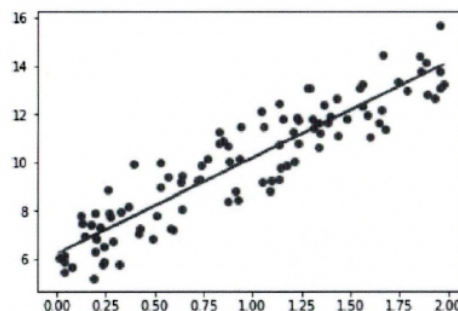
$$\frac{\partial R(w)}{\partial w_0} = \frac{2}{N} \sum_{i=1}^N -(y_i - (w_0 + w_1 x_i)) = -\frac{2}{N} \sum_{i=1}^N (\text{실제값}_i - \text{예측값}_i)$$

## 🔧 경사 하강법 프로세스

1.  $w_0, w_1$ 을 임의로 설정, 비용 계산
2.  $w_0, w_1$  업데이트 후 다시 비용 계산
3. 비용이 감소하는 방향으로 주어진 횟수만큼 반복

## 🔧 파이썬 코드 구현 (핵심 흐름)

- 데이터 생성:  $y = 4x + 6$  주위로 무작위 분포
- 비용 함수 `get_cost()`
- 경사 하강법 `gradient_descent()`
  - 초기  $w_0, w_1 = 0$
  - `iters` 만큼 반복 업데이트
- 예측:  $\text{ypred} = \text{np.dot}(X, w_1.T) + w_0$
- 최종 결과:  $w_1: 4.022, w_0: 6.162$ , 예측 오류 약 0.9935



## 🔧 확률적 경사 하강법(SGD)

- 전체 데이터가 아닌 일부(batch)만 사용해 w 업데이트.
- 일반 경사 하강법보다 빠름.
- (미니 배치) 확률적 경사 하강법으로도 유사한 결과 도출 가능.

### 🔧 다중 피쳐 확장

- 피쳐 1개 →  $y_{\text{pred}} = w_0 + w_1x$
- 피쳐 여러 개 →  $y_{\text{pred}} = w_0 + w_1 \times 1 + w_2 \times 2 + \dots + w_{100} \times 100$
- 벡터화:
  - 피쳐 1개 →  $y = \text{np.dot}(X, w_1.T) + w_0$
  - 피쳐 여러 개 →  $y' = \text{np.dot}(X_{\text{mat}}, W^T) + w_0$ 
    - N : 데이터 개수
    - M : 피쳐 M개
    - Xmat : 입력 행렬
    - $w_1, w_2, \dots, w_{100}$  을 W : 회귀 계수
    - $w_0$  포함 위해 Xmat 첫 열에 1 추가 → 선형 대수로 간단히 계산 가능

$$\hat{Y} = X_{\text{mat}} * W^T$$

Diagram illustrating the matrix multiplication for linear regression with multiple features. The input matrix  $X_{\text{mat}}$  has dimensions  $N \times (M+1)$ , where  $N$  is the number of samples and  $M+1$  is the number of features (including the bias term  $w_0$ ). The weight matrix  $W$  has dimensions  $(M+1) \times 1$ . The resulting predicted values  $\hat{Y}$  are a column vector of size  $N$ .

The diagram shows the matrix  $X_{\text{mat}}$  with columns labeled Feat 0, Feat 1, Feat 2, ..., Feat M. The first column (Feat 0) contains ones, representing the bias term  $w_0$ . The other columns contain the feature values  $x_{11}, x_{12}, \dots, x_{1M}$  for the first sample, and so on for subsequent samples up to  $x_{n1}, x_{n2}, \dots, x_{nM}$ .

The weight vector  $W$  is shown as  $[w_0, w_1, w_2, \dots, w_M]^T$ . The bias term  $w_0$  is highlighted with a box and labeled "내적" (inner product).

## ✅ 04 사이킷런 LinearRegression을 이용한 보스턴 주택 가격 예측

### 🔧 LinearRegression 클래스 - Ordinary Least Squares

- `LinearRegression` 클래스는 RSS(Residual Sum of Squares) 를 최소화하는 OLS(Ordinary Least Squares) 방식으로 회귀 계수를 계산합니다.
- `fit(X, y)` 를 호출하면 회귀 계수(`coef_`) 와 절편(`intercept_`) 이 저장됩니다.
- 주요 입력 파라미터:
  - `fit_intercept` : 절편 계산 여부 (기본 True)
  - `normalize` : 입력 데이터 정규화 여부 (기본 False)
- 주요 속성:
  - `coef_` : 회귀 계수 (Shape: (Target 개수, 피쳐 개수))

- `intercept_`: 절편 값

```
class sklearn.linear_model.LinearRegression(fit_intercept=True, normalize=False, copy_X=True, n_jobs=None)
```

## 🔧 다중 공선성 (Multi-collinearity)

- Ordinary Least Squares 기반 회귀 계수 계산은 입력 피처의 독립성에 큰 영향을 받습니다.
- 피처 간 상관관계가 매우 높은 경우, 분산이 커지고 오류에 민감해집니다.
- 이를 다중 공선성 (multi-collinearity) 문제라고 부릅니다.
- 해결 방법:
  - 상관관계가 높은 피처를 제거하거나
  - 규제(regularization) 를 적용하거나
  - 피처가 매우 많으면 PCA (차원 축소) 를 고려할 수 있습니다.

## 🔧 회귀 평가 지표

- 회귀 평가 지표는 실제 값과 예측값의 차이를 기반으로 합니다.
- 단순 합은 +와 -가 상쇄되므로, 절댓값이나 제곱, 또는 루트 씌운 값으로 측정합니다.
- 주요 지표

| 평가 지표          | 설명                                                                                             | 수식                                                           |
|----------------|------------------------------------------------------------------------------------------------|--------------------------------------------------------------|
| MAE            | Mean Absolute Error(MAE)이며 실제 값과 예측값의 차이를 절댓값으로 변환해 평균한 것입니다.                                  | $MAE = \frac{1}{n} \sum_{i=1}^n  Y_i - \hat{Y}_i $           |
| MSE            | Mean Squared Error(MSE)이며 실제 값과 예측값의 차이를 제곱해 평균한 것입니다.                                         | $MSE = \frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2$         |
| RMSE           | MSE 값은 오류의 제곱을 구하므로 실제 오류 평균보다 더 커지는 특성이 있으므로 MSE에 루트를 씌운 것이 RMSE(Root Mean Squared Error)입니다. | $RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (Y_i - \hat{Y}_i)^2}$ |
| R <sup>2</sup> | 분산 기반으로 예측 성능을 평가합니다. 실제 값의 분산 대비 예측값의 분산 비율을 지표로 하며, 1에 가까울수록 예측 정확도가 높습니다.                   | $R^2 = \frac{\text{예측값 Variance}}{\text{실제값 Variance}}$      |

## 🔧 사이킷런(Scikit-Learn) 관련

- `mean_squared_error()` 함수 사용:
  - `squared=True` → MSE 계산
  - `squared=False` → RMSE 계산
- 각 지표에 해당하는 `scoring` 파라미터 값:

|       |     |              |
|-------|-----|--------------|
| 평가 방법 | API | scoring 적용 값 |
|-------|-----|--------------|

|      |                                           |                               |
|------|-------------------------------------------|-------------------------------|
| MAE  | metrics.mean_absolute_error               | 'neg_mean_absolute_error'     |
| MSE  | metrics.mean_squared_error                | 'neg_mean_squared_error'      |
| RMSE | metrics.mean_squared_error(squared=False) | 'neg_root_mean_squared_error' |
| MSLE | metrics.mean_squared_log_error            | 'neg_mean_squared_log_error'  |
| R2   | metrics.r2_score                          | 'r2'                          |

scoring 값에는 neg\_ 접두어가 붙음:

- Scoring 함수는 점수가 클수록 좋은 모델로 인식하기 때문에, 오류 기반 지표는 -1을 곱해 음수로 반환합니다.
  - 예: MAE는 원래  $10 > 1$ 이지만, neg 처리하면  $-1 > -10$

## 데이터 로드와 확인

- 사이킷런의 load\_boston() 으로 보스턴 주택 가격 데이터 세트를 로드하고 DataFrame으로 변환합니다.
- bostonDF.info() 로 Null 값이 없고 모든 피처가 float 형임을 확인합니다.

## 피처별 PRICE 영향 시각화

- RM, ZN, INDUS, NOX, AGE, PTRATIO, LSTAT 총 8개 피처에 대해 가격 변화 관찰.
- seaborn.regplot() 으로 각 피처와 PRICE 간의 산점도 및 선형 회귀선을 시각화합니다.
- matplotlib.subplots() 를 이용해 2행 4열로 8개 그래프를 그립니다.
- 관찰 결과:
  - RM: 양 방향 선형성(Positive Linearity) → 방 개수 많을수록 가격 상승
  - LSTAT: 음 방향 선형성(Negative Linearity) → 하위 계층 비율 낮을수록 가격 상승

## 모델 학습 및 평가

- train\_test\_split() 로 학습/테스트 세트 분리 후 학습 및 예측 수행.
- metrics.mean\_squared\_error() 와 metrics.r2\_score() 로 MSE와 R2 Score 측정.
- 모델 속성:
  - intercept\_ : 절편
  - coef\_ : 회귀 계수
- coef\_ 를 피처별로 매핑해 pandas.Series 로 변환 후 sort\_values() 로 높은 값 순으로 출력.
- RM 피처의 회귀 계수가 가장 큼.
- NOX 피처는 음의 방향으로 회귀 계수가 크게 나타남.

## 교차 검증과 RMSE 계산

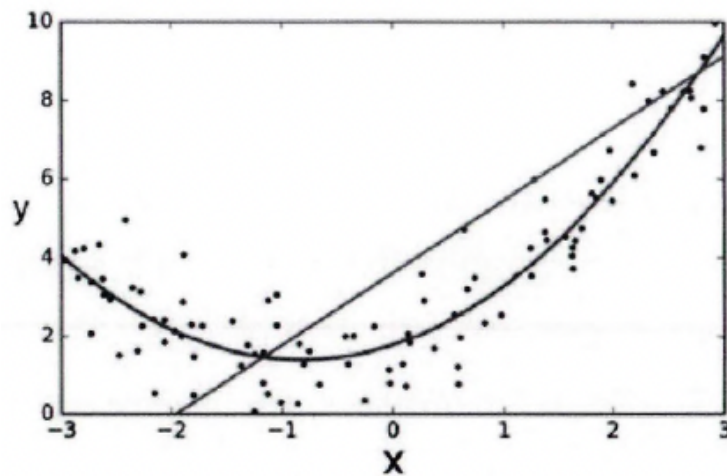
- cross\_val\_score() 를 사용해 5개 폴드 세트에서 교차 검증 수행.
- scoring="neg\_mean\_squared\_error" 로 지정해 MSE의 음수 반환을 받고, 반환된 음수값에 다시 1을 곱해 원래 MSE를 구한 후, numpy.sqrt() 를 적용해 RMSE를 계산합니다.

- 결과:
  - 5개 폴드 세트 평균 RMSE는 약 5.829

## ✅ 05 다항 회귀와 과(대)적합/과소적합 이해

### 🔧 다항 회귀 개념

- 기존 회귀:  $y = w_0 + w_1x_1 + w_2x_2 + \dots + w_nx_n$  (1차 방정식 형태)
- 다항 회귀: 독립변수와 종속변수의 관계를 2차, 3차 방정식 같은 다항식으로 표현
  - 예:  $y = w_0 + w_1x_1 + w_2x_2 + w_3x_1x_2 + w_4x_1^2 + w_5x_2^2$



〈 주어진 데이터 세트에서 다항 회귀가 더 효과적임 〉

- 다항 회귀는 여전히 선형 회귀입니다.
- 선형/비선형 기준은 회귀 계수가 선형/비선형인지에 따라 결정합니다.
- 다항식 변수를 새로 만든 후, 선형 회귀를 적용할 수 있습니다.
- 단순 선형 회귀보다 데이터의 복잡한 패턴을 더 잘 예측할 수 있습니다.

### 🔧 사이킷런에서 다항 회귀 구현

- 사이킷런은 다항 회귀 전용 클래스를 제공하지 않습니다.
- 대신, `PolynomialFeatures` 클래스로 다항식 피처를 만들고, `LinearRegression`을 적용합니다.
- `PolynomialFeatures` :
  - `degree` 파라미터로 다항식 차수 설정
  - `fit()`, `transform()` 메서드 사용

### 🔧 다항 피처 변환 예시

|               |              |
|---------------|--------------|
| 원래 피처 [x1,x2] | 변환된 2차 다항 피처 |
|---------------|--------------|



|        |                    |
|--------|--------------------|
| [0, 1] | [1, 0, 1, 0, 0, 1] |
| [2, 3] | [1, 2, 3, 4, 6, 9] |

- 변환된 피처에 선형 회귀를 적용하면 다항 회귀를 구현할 수 있습니다.

### 3차 다항 회귀 구현 예시

- 3차 함수 식 예시:  $y = 1 + 2 \cdot x_1 + 2 \cdot x_1^2 + 4 \cdot x_2^3$
- `PolynomialFeatures(degree=3)` 로 3차 다항 피처 생성
- 변환 후 10개의 다항 피처가 만들어짐
- 선형 회귀 적용 → 회귀 계수 10개 생성
- 실제 회귀 계수는 원래 식과 완벽히 일치하지는 않지만, 가까운 근사를 합니다.

### Pipeline으로 다항 회귀 간단히 구현

- Pipeline 객체를 사용해:
  - 피처 변환 (`PolynomialFeatures`)
  - 선형 회귀 (`LinearRegression`)
- 두 단계를 한 번에 연결해 코드 작성이 더 명료해집니다.

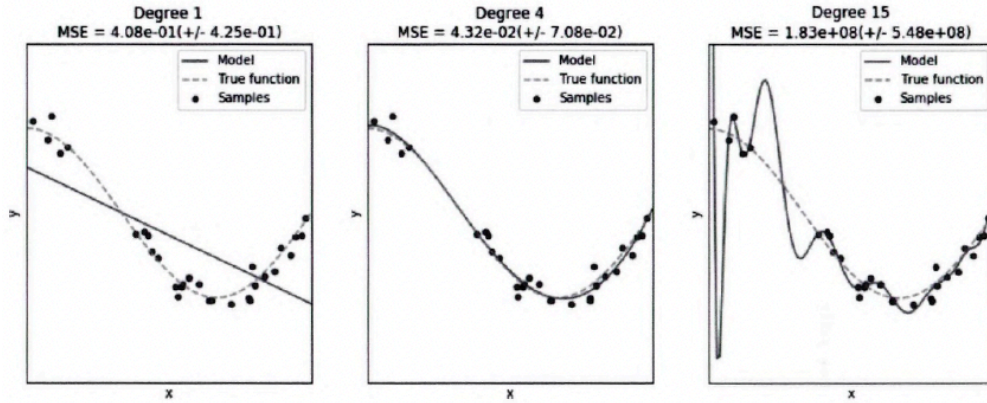
---

### 다항 회귀를 통한 과소적합(Underfitting)과 과적합(Overfitting) 이해

- 다항 회귀는 피처 간 직선 관계가 아닌 복잡한 다항 관계를 모델링할 수 있습니다.
- 다항식의 차수(degree)가 높아질수록:
  - 복잡한 패턴까지 모델링 가능
  - 그러나 과적합(overfitting) 위험 증가
  - 학습 데이터에는 잘 맞지만 테스트 데이터 예측 성능은 나빠질 수 있음

### 예제 설명

- 데이터:
  - 피처 X 는 0~1 구간의 30개 임의 값
  - 타겟 y는 X의 코사인 값에 잡음(Noise) 추가
- 목표:
  - 다항식 차수를 바꿔가며 (1, 4, 15)
  - 예측 곡선과 예측 성능(MSE)을 비교



#### • 다항식 차수별 결과 비교

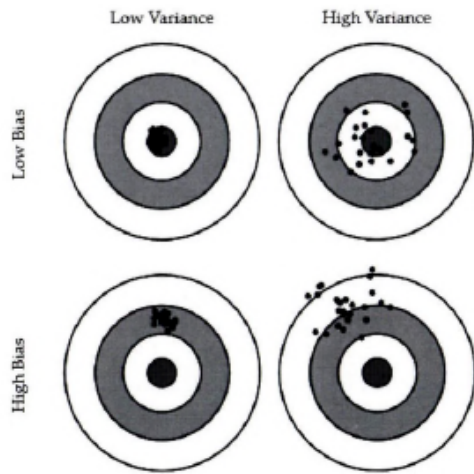
| 차수 (Degree) | 설명                  | 결과                                                             |
|-------------|---------------------|----------------------------------------------------------------|
| Degree 1    | 단순 선형 회귀 (직선)       | 데이터 패턴을 반영 못함 → <b>과소적합</b> (MSE $\approx 0.41$ )              |
| Degree 4    | 코사인 곡선과 비슷한 형태      | 잡음을 제외한 데이터 패턴을 잘 예측 → <b>최적 모델</b> (MSE $\approx 0.04$ )      |
| Degree 15   | 학습 데이터 잡음까지 지나치게 학습 | 테스트 데이터와는 다른 이상한 곡선 → <b>과적합</b> (MSE $\approx 182581084.83$ ) |

- 좋은 예측 모델은:
  - Degree 1처럼 너무 단순한 과소적합 모델도 아니고,
  - Degree 15처럼 너무 복잡한 과적합 모델도 아님
- 학습 데이터 패턴을 적절히 반영하면서 복잡도도 적당한 균형 잡힌(Balanced) 모델이어야 합니다.

#### 🔧 편향-분산 트레이드오프 (Bias-Variance Trade-off) 개념

- 편향(Bias): 모델이 너무 단순해서 실제 데이터 패턴을 잘 반영하지 못하는 성향.
  - 예: Degree 1 모델 → 고편향(High Bias)
- 분산(Variance): 모델이 너무 복잡해서 학습 데이터의 작은 변동까지 민감하게 반응하는 성향.
  - 예: Degree 15 모델 → 고분산(High Variance)
- 머신러닝 모델은 편향과 분산을 동시에 조절해야 함.

#### 🔧 양궁 과녁 비유

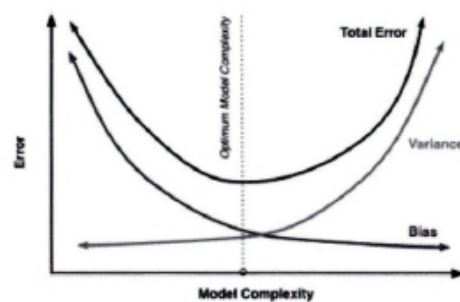


| 구분      | 의미                                        |
|---------|-------------------------------------------|
| 저편향/저분산 | 예측 결과가 실제 결과에 정확하고 안정적으로 집중. (아주 이상적인 경우) |
| 저편향/고분산 | 예측 결과가 실제 주변에 있지만 퍼져 있음. (예측 변동 큼)        |
| 고편향/저분산 | 엉뚱한 위치지만 예측은 한 곳에 몰림. (일관된 오류)            |
| 고편향/고분산 | 엉뚱한 위치에 퍼져 있음. (예측도 부정확하고 불안정)            |

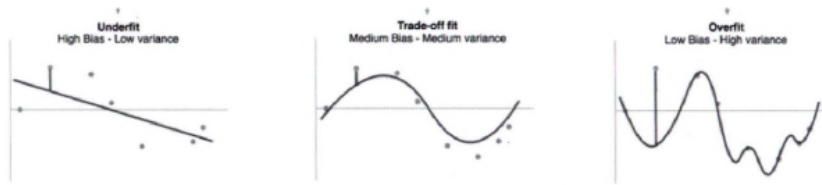
### 🔧 편향과 분산 관계

- 편향이 높으면 분산이 낮아지고 → 과소적합
- 분산이 높으면 편향이 낮아지고 → 과적합
- 편향을 낮추다 보면 분산이 올라가고,
- 분산을 조절하다 보면 편향이 올라가는 Trade-off(균형) 관계에 있음.

### 🔧 Total Error(전체 오류) 변화



- 편향이 너무 높으면 → 전체 오류도 높음
- 편향을 낮추면 → 전체 오류가 줄어듦
- 하지만 분산이 너무 높아지면 → 전체 오류가 다시 증가
- 편향-분산이 균형을 이루는 지점(골디락스 지점)에서 전체 오류가 가장 낮음.
- 결론



- 높은 편향/낮은 분산 → 과소적합 (Underfitting) 경향
- 낮은 편향/높은 분산 → 과적합 (Overfitting) 경향
- 편향과 분산을 적절히 트레이드오프 하여 오류를 최소화하는 것이 가장 좋은 머신러닝 모델을 만드는 방법이다.

## ✓ 06 규제 선형 모델 - 릿지, 라쏘, 엘라스틱넷

### 🔧 좋은 회귀 모델의 특징

- 데이터에 적절히 적합되면서도 회귀 계수(W)가 기하급수적으로 커지지 않도록 제어해야 함.
- 너무 단순하면 → 과소적합
- 너무 복잡하고 계수가 크면 → 과적합

### 🔧 기존 선형 모델 비용 함수의 한계

- 기존 비용 함수는 RSS(Residual Sum of Squares)만 최소화.
- 학습 데이터에만 잘 맞추다 보니 회귀 계수가 커지고, 테스트 데이터 성능이 나빠지는 문제 발생.

### 🔧 규제를 적용한 새로운 비용 함수

$$\text{비용 함수 목표} = \text{Min}(\text{RSS}(W) + \alpha * \|W\|_2^2)$$

- $\text{RSS}(W)$ : 잔차 오류 최소화
- $\alpha * \|W\|^2$ : 회귀 계수 크기에 대한 페널티 추가
- $\alpha$ : 데이터 적합과 회귀 계수 제어의 균형을 맞추는 튜닝 파라미터

### 🔧 $\alpha$ (alpha) 값의 역할

| alpha 값             | 효과                                    |
|---------------------|---------------------------------------|
| $\alpha = 0$        | 기존과 동일하게 RSS만 최소화 (규제 없음)             |
| $\alpha$ 매우 큼 (무한대) | 회귀 계수 W를 0에 가깝게 만들어야 비용이 최소화됨 (규제 강함) |

alpha 값을 키우면 → 회귀 계수 크기 줄어듦 → 과적합 방지

alpha 값을 작게 하면 → 학습 데이터 적합을 좀 더 강화할 수 있음

### 🔧 규제(Regularization)의 개념

- 비용 함수에 페널티를 추가해 회귀 계수 크기를 조정하는 방법
- 과적합(overfitting) 문제를 개선할 수 있음

## 🔧 규제 종류

| 규제 방식 | 설명                 | 적용한 회귀 이름 |
|-------|--------------------|-----------|
| L2 규제 | $\alpha *   W  ^2$ | 릿지(Ridge) |
| L1 규제 | $\alpha *   W  ^1$ | 라쏘(Lasso) |

- L2 규제 : W의 제곱에 대해 패널티를 부여하는 방식
- L1 규제: W의 절대값에 대해 패널티를 부여하는 방식
  - L1 규제는 중요하지 않은 계수를 0으로 만들어서 모델을 더 단순하게 만들 수 있음.

## 🔧 릿지 회귀

- 사이킷런은 Ridge 클래스를 통해 릿지 회귀를 구현합니다.
- Ridge 클래스의 주요 생성 파라미터는 alpha로, L2 규제 계수에 해당합니다.
- 릿지 회귀는 L2 규제를 적용해 회귀 계수 크기를 줄이면서 과적합을 방지합니다.

## 🔧 보스턴 주택 가격 예측

- 기존 LinearRegression 예제의 X\_data와 y\_target을 그대로 사용.
- 5개 폴드 교차검증으로 평균 RMSE 평가.

| 모델                       | 평균 RMSE |
|--------------------------|---------|
| LinearRegression (규제 없음) | 5.829   |
| RidgeRegression (규제 적용)  | 5.518   |

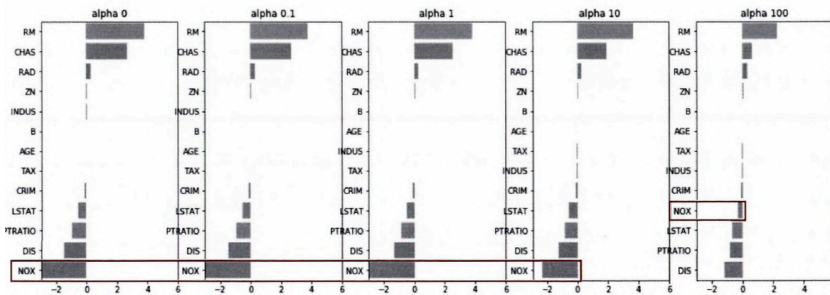
릿지 회귀가 규제 덕분에 **더 낮은 RMSE**를 보여줌.

## 🔧 alpha 값 변화 실험

- alpha 값을 0, 0.1, 1, 10, 100으로 변화시키며,
- 5 폴드 평균 RMSE와 회귀 계수(coef\_) 변화를 관찰.
- 결과:
  - alpha = 100일 때 평균 RMSE가 5.330으로 가장 좋음.

## 🔧 alpha 값에 따른 회귀 계수 변화

- alpha 증가 → 회귀 계수 크기 감소
- 특히 NOX 피처의 경우, alpha 증가에 따라 회귀 계수가 크게 줄어듦.
- alpha 값별 회귀 계수 변화를 DataFrame에 저장하고, Seaborn 가로 막대 그래프로 시각화함.



## 🔧 릿지 회귀 특징

- alpha 값이 커질수록 회귀 계수 크기는 지속적으로 작아진다.
- 하지만 릿지 회귀는 회귀 계수를 0으로 만들지는 않는다.

## 🔧 라쏘 회귀

- L1 규제를 선형 회귀에 적용한 것이 라쏘(Lasso) 회귀입니다.
- 비용 함수 목표:  

$$RSS(W) + \alpha \cdot ||W||_1$$
- 특징:
  - 불필요한 회귀 계수를 급격히 줄여서 0으로 만들.
  - 자연스럽게 피쳐 선택(Feature Selection) 기능을 가짐.

## 🔧 사이킷런 Lasso 클래스

- Lasso 클래스를 통해 라쏘 회귀를 구현합니다.
- 주요 생성 파라미터는 alpha (L1 규제 계수).

## 🔧 실험: alpha 값 변화

- 이전 릿지 회귀 실험과 비슷하게, alpha 값을 변화시키면서 RMSE와 회귀 계수(coef\_) 변화를 관찰합니다.
- 함수를 따로 생성:

```
get_linear_reg_eval(model_name, params, X_data, y_target)
```

- 인자: 모델명(lasso), alpha 값 리스트, 피쳐 데이터, 타겟 데이터
- 기능: alpha별 평균 RMSE 출력 + 회귀 계수 DataFrame 반환
- 실험 결과
  - alpha 리스트: [0.07, 0.1, 0.5, 1, 3]
- 결과:

| 모델                 | 평균 RMSE       |
|--------------------|---------------|
| Lasso (alpha=0.07) | 5.612 (가장 좋음) |

|                          |       |
|--------------------------|-------|
| Ridge                    | 5.518 |
| LinearRegression (규제 없음) | 5.829 |

Lasso는 LinearRegression보다는 성능이 좋지만, Ridge보다는 약간 낮음.

### 🔧 회귀 계수 변화 특징

- alpha 값이 커질수록, 일부 피처의 회귀 계수가 0으로 바뀜.
- 예시:
  - NOX 피처: alpha=0.07부터 계수 0
  - 이후 INDUS, CHAS 피처들도 계수 0

회귀 계수가 0이 된 피처는 회귀식에서 제외되어 자동으로 피처 선택 효과를 얻을 수 있음.

### 🔧 엘라스틱넷 회귀

- 엘라스틱넷은 L2 규제와 L1 규제를 결합한 회귀입니다.
- 비용 함수 목표:

$$\text{Min}(\text{RSS}(W) + \alpha_2 * ||W||_2 + \alpha_1 * ||W||_1)$$

L2 규제(Ridge) + L1 규제(Lasso)를 모두 적용.

### 🔧 왜 엘라스틱넷의 필요성

- 라쏘 회귀는 상관관계가 높은 피처들 중에서 하나만 선택하고 나머지 피처들은 회귀 계수를 0으로 만들기 쉽다.
- 이 과정에서 회귀 계수가 급격히 변동할 수도 있다.
- 엘라스틱넷은 이를 완화하기 위해 L2 규제를 추가하여 안정성을 높인다.
- 단점:
  - L1과 L2 규제를 모두 적용하기 때문에 **수행시간이 오래 걸릴 수 있음**.

### 🔧 사이킷런 ElasticNet 클래스

- ElasticNet 클래스를 통해 엘라스틱넷 회귀를 구현합니다.

| 파라미터                  | 설명                   |
|-----------------------|----------------------|
| <code>alpha</code>    | a+b, 규제 강도의 합        |
| <code>l1_ratio</code> | a/(a+b), L1 규제 비율 설정 |

l1\_ratio에 따라 의미가 달라짐:

- `l1_ratio = 0` → L2 규제만 (Ridge)
- `l1_ratio = 1` → L1 규제만 (Lasso)

### 🔧 실험: alpha 값 변화

- 이전 릿지, 라쏘 회귀와 유사하게, alpha 값을 변화시키면서 RMSE와 회귀 계수(coef\_) 변화를 관찰.

- `get_linear_reg_eval()` 함수를 재사용.
- 주의:
  - ElasticNet 객체 생성 시 `l1_ratio=0.7`로 고정.
  - 실험에서는 `alpha` 값만 변화시키며 비교.

## 🔧 선형 회귀 모델을 위한 데이터 변환

- 선형 회귀 모델은
  - 피처와 타깃값 간에 선형 관계가 있다고 가정.
  - 정규 분포(평균을 중심으로 한 종 모양 분포)를 선호.
- 타깃값이나 피처값 분포가 왜곡(Skew) 되어 있으면  
→ 예측 성능이 떨어질 수 있음.  
따라서, 필요시 스케일링/정규화 또는 로그 변환을 수행.

## 🔧 피처 데이터 세트 변환 방법

| 방법                            | 설명                                            |
|-------------------------------|-----------------------------------------------|
| 1. StandardScaler             | 평균 0, 분산 1인 <b>표준 정규 분포</b> 로 변환              |
| 2. MinMaxScaler               | 최솟값 0, 최댓값 1로 <b>정규화</b>                      |
| 3. 다항 특성 변환                   | 위 1번/2번을 수행한 후, <b>다항 특성 추가</b>               |
| 4. 로그 변환 (Log Transformation) | <code>np.log1p()</code> 로 원본 값에 1을 더한 뒤 로그 변환 |

## 🔧 로그 변환

- 데이터 왜곡이 심할 때 가장 많이 쓰이고, 선형 회귀 성능 향상에 가장 효과적인 경우가 많음.
- 타깃값은 일반적으로 로그 변환을 적용.
- 타깃값을 변환하면 원복이 어려울 수 있지만, 왜곡된 타깃 분포를 바로잡아 예측 성능을 크게 향상시킬 수 있음.

## 🔧 변환 실습: 보스턴 주택가격 예제

- 다양한 방법으로 피처 데이터 변환 후 **Ridge 회귀** 적용.
- `get_scaled_data()` 함수로 변환 방법 선택:
  - `'Standard'`, `'MinMax'`, `'Log'`
- 5가지 데이터 변환 조합:

| 변환 조합              | 설명                   |
|--------------------|----------------------|
| (None, None)       | 원본 데이터 사용            |
| ('Standard', None) | 표준 정규 분포 변환만         |
| ('Standard', 2)    | 표준 정규 분포 + 2차 다항식 변환 |
| ('MinMax', None)   | 최솟값/최댓값 정규화만         |



|               |                         |
|---------------|-------------------------|
| ('MinMax', 2) | 최솟값/최댓값 정규화 + 2차 다항식 변환 |
| ('Log', None) | 로그 변환만                  |

- 결과

| 변환 유형                | alpha 값      |              |              |              |
|----------------------|--------------|--------------|--------------|--------------|
|                      | alpha=0.1    | alpha=1      | alpha=10     | alpha=100    |
| 원본 데이터               | 5.788        | 5.653        | 5.518        | 5.330        |
| 표준 정규 분포             | 5.826        | 5.803        | 5.637        | 5.421        |
| 표준 정규 분포 + 2차 다항식    | 8.827        | 6.871        | 5.485        | <b>4.634</b> |
| 최솟값/최댓값 정규화          | 5.764        | 5.465        | 5.754        | 7.635        |
| 최솟값/최댓값 정규화 + 2차 다항식 | 5.298        | <b>4.323</b> | 5.185        | 6.538        |
| 로그 변환                | <b>4.770</b> | <b>4.676</b> | <b>4.836</b> | 6.241        |

| 변환 방법                   | 주요 결과                        |
|-------------------------|------------------------------|
| 표준 정규 분포 / 최댓값-최솟값 정규화만 | 성능 향상 없음                     |
| 표준 정규 분포 + 2차 다항 변환     | alpha=100에서 RMSE 4.634       |
| 최댓값-최솟값 정규화 + 2차 다항 변환  | alpha=1에서 RMSE 4.323         |
| 로그 변환                   | alpha=0.1, 1, 10 모두 좋은 성능 향상 |

로그 변환이 가장 꾸준하고 좋은 성능 향상을 보임.

## ✓ 07 로지스틱 회귀

### 🔧 로지스틱 회귀 개념

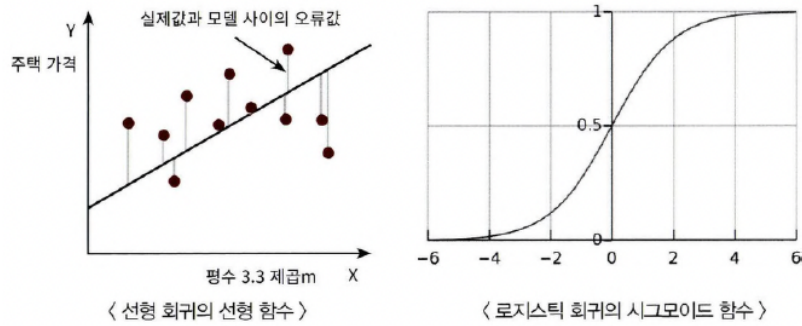
- 로지스틱 회귀는 선형 회귀 방식을 분류에 적용한 알고리즘입니다.
- 선형 모델이지만, 결과를 시그모이드 함수로 변환해 0과 1 사이 확률로 예측합니다.
- 시그모이드 함수:

- $y = 1/(1+e^{-x})$

### 🔧 특징:

- x 값이 커지면 y는 1에 근접
- x 값이 작아지면 y는 0에 근접
- x=0일 때 y=0.5

### 🔧 선형 회귀 vs 로지스틱 회귀



- 선형 회귀는 연속형 값 예측.
- 로지스틱 회귀는 0/1 분류. 시그모이드 함수를 통해 S자 형태로 0과 1을 정확하게 분류 가능.
- 종양 예제:
  - 선형 회귀: 종양 크기별 악성 예측이 부정확
  - 로지스틱 회귀: 시그모이드 커브를 통해 정확한 분류 가능

## 🔧 사이킷런 LogisticRegression 클래스

- 다양한 최적화 solver 제공:

| solver      | 특징                   |
|-------------|----------------------|
| lbfgs (기본값) | 메모리 절약, 병렬 최적화 가능    |
| liblinear   | 작은 데이터에 효과적, 병렬 불가   |
| newton-cg   | 정교하지만 느림             |
| sag         | 대용량 데이터에 빠름 (SGD 기반) |
| saga        | sag 변형 + L1 규제 지원    |

일반적으로 lbfgs 또는 liblinear 선택.

## 🔧 위스콘신 유방암 데이터 예제

- `load_breast_cancer()` 로 데이터 로드
- 표준 스케일링 후 `train_test_split()`
- 결과:
  - solver=lbfgs → 정확도 0.977, ROC-AUC 0.995
  - solver=liblinear, sag, saga → 정확도 0.982, ROC-AUC 0.995
- 데이터가 작아서 solver별 성능 차이는 미미.

## 🔧 주요 하이퍼파라미터

| 하이퍼파라미터 | 설명                    |
|---------|-----------------------|
| penalty | 규제 방식 (L2 기본, L1도 가능) |

|          |                                         |
|----------|-----------------------------------------|
| C        | 규제 강도 조절 ( $C=1/\alpha$ , C 작을수록 규제 강함) |
| max_iter | 최적화 최대 반복 횟수 설정                         |

- liblinear, saga → L1/L2 모두 가능
- lbfgs, newton-cg, sag → L2만 가능

### GridSearchCV로 최적화 실험

- solver, penalty, C 값 최적화
- 가장 좋은 성능:

| 항목      | 값         |
|---------|-----------|
| solver  | liblinear |
| penalty | L2        |
| C       | 0.1       |
| 평균 정확도  | 0.979     |

### 로지스틱 회귀 장점

- 가볍고 빠르다
- 이진 분류 기본 모델로 많이 사용
- 희소한 데이터(텍스트 분류 등)에서도 성능 좋음

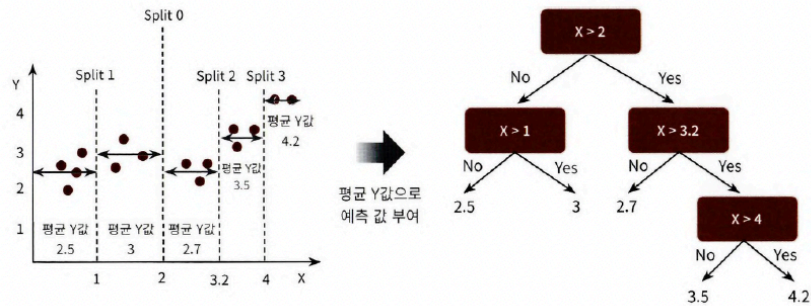
## 08 회귀 트리

### 회귀 트리 개념

- 선형 회귀는 회귀 계수를 선형 결합해 결과를 예측.
- 회귀 트리는 회귀 함수를 따르지 않고, 트리(Decision Tree)를 기반으로 예측을 수행. 분류 트리와 비슷하지만,
  - 분류 트리: 리프 노드에서 클래스 레이블 결정
  - 회귀 트리: 리프 노드에서 속한 데이터 값들의 평균을 예측 값으로 결정

### 회귀 트리 생성 방법

- 트리 분할은 분류 트리와 비슷하게 진행.
- 리프 노드에 도달하면 해당 노드 데이터들의 평균을 최종 예측값으로 사용.
- CART 알고리즘 기반:
  - CART(Classification And Regression Trees): 분류와 회귀 모두 지원하는 트리 알고리즘.
- 간단한 예제



- X와 Y 데이터 준비
- 루트 노드에서 첫 번째 분할 (Split 0)
- 자식 노드들 추가 분할 (Split 1, Split 2)
- 더 세부 분할 (Split 3)
- 분할 완료 → 리프 노드 생성
- 리프 노드마다 Y 평균값 계산 → 예측값 할당 (2.5, 3, 2.7, 3.5, 4.2)

### 🔧 다양한 트리 기반 회귀 모델

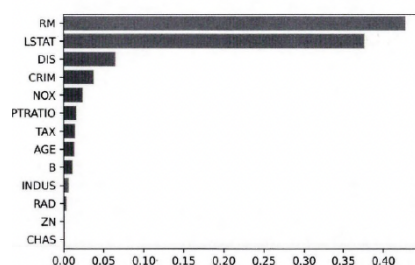
| 알고리즘              | 회귀 Estimator 클래스                       | 분류 Estimator 클래스                        |
|-------------------|----------------------------------------|-----------------------------------------|
| Decision Tree     | <code>DecisionTreeRegressor</code>     | <code>DecisionTreeClassifier</code>     |
| Gradient Boosting | <code>GradientBoostingRegressor</code> | <code>GradientBoostingClassifier</code> |
| XGBoost           | <code>XGBRegressor</code>              | <code>XGBClassifier</code>              |
| LightGBM          | <code>LGBMRegressor</code>             | <code>LGBMClassifier</code>             |

사이킷런, XGBoost, LightGBM 모두 회귀와 분류 모두 지원.

### 🔧 보스턴 주택 가격 예제

- `RandomForestRegressor` 를 포함한 다양한 Regressor로 보스턴 주택 가격 예측 수행.
- `get_model_cv_prediction()` 함수를 통해 교차 검증 평균 RMSE를 계산.

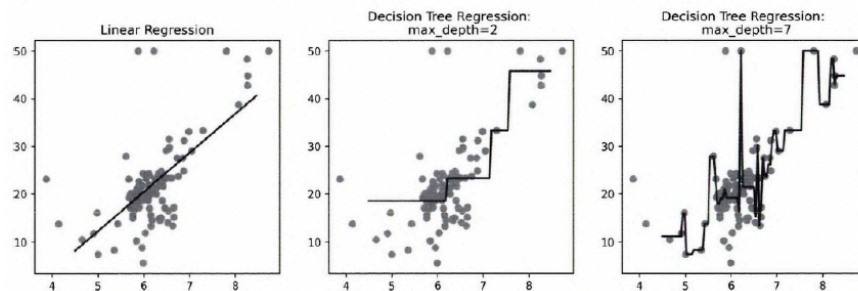
### 🔧 회귀 트리와 선형 회귀 비교



- 회귀 트리는 회귀 계수(coef\_) 제공 X. 대신 feature\_importances\_ 속성을 통해 피쳐 중요도를 제공.

## 🔧 회귀 트리 Regressor와 선형 회귀 비교 시각화

- max\_depth에 따른 회귀 트리 변화 실험
- 데이터 준비:
  - 보스턴 데이터에서 RM (방 개수)와 PRICE(가격)만 사용.
  - 100개 샘플만 선택.
- 시각화:



| 모델                                  | 설명                     |
|-------------------------------------|------------------------|
| LinearRegression                    | 직선 형태의 회귀선             |
| DecisionTreeRegressor (max_depth=2) | 비교적 단순한 계단 형태          |
| DecisionTreeRegressor (max_depth=7) | 복잡한 계단 형태 (과적합 가능성 높음) |

max\_depth=7일 경우, 이상치(outlier)까지 학습하여 과적합 경향.